Kriptovaluták elméleti háttere

Ladóczki Bence
docens
Távközlési és Mesterséges Intelligencia Tanszék (TMIT)
Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar (VIK)
ladoczki.bence@vik.bme.hu

2026. október 14.

Tartalomjegyzék

1 Konszenzus mechanizmusok **3**
1.1 A blokklánc alapstruktúrája . . . 3
1.2 Proof-of-work (munkabizonyíték) . . . 4
1.3 Proof-of-stake . . . 4
1.4 Egyéb konszenzus mechanizmusok . . . 5
1.4.1 Practical Byzantine Fault Tolerance (PBFT) . . . 5
1.4.2 Tendermint és az FLP lehetetlenség . . . 6
1.5 Proof-of-spacetime . . . 7
1.5.1 A proof-of-space alapötlete . . . 7
1.5.2 Hellman trade-off és a DES példa . . . 7
1.5.3 Beyond Hellman: modern proof-of-space konstrukciók . . . 8

1 Konszenzus mechanizmusok

Az előző fejezetekben megismertük az elosztott rendszerek elméleti alapjait: Neumann redundancia-elvét, Lamport logikai óráit és állapotgép-replikációját, valamint a bizánci tábornokok problémáját. Ebben a fejezetben azt vizsgáljuk, hogy ezek az elméleti eredmények hogyan valósulnak meg a blokklánc-technológiában. A fejezet a következő alapvető kérdésekre keresi a választ:

- Mi az a blokklánc, és hogyan működik?
- Mekkora késleltetést tolerálhat a rendszer?
- Hogyan definiálják a többséget?
- Hány csaló (bizánci hibás csomópont) ellen lehet védekezni?
- Hogyan fogadják el az állapotváltozásokat?
- Milyen korlátokat adhatunk a biztonságos működéshez?
- Mi történik, ha alternatív valóságok (elágazások) jelennek meg?
- Milyen centralizáló erők léphetnek fel?
- Hogyan lehet a támadók ellen védekezni, és milyen pénzügyi ösztönzőkkel?

1.1 A blokklánc alapstruktúrája

Tegyük fel, hogy hálózatba kötött számítógépeink vannak, amelyek egy peer-to-peer (P2P) hálózatot alkotnak. Egyetlen központi szerver sincs – minden csomópont egyenrangú. Amikor valaki tranzakciót szeretne küldeni (például kriptovalutát utalni), a folyamat a következő lépésekből áll:

1. A tranzakció blokk formájában jelenik meg (egy blokk több tranzakciót is tartalmazhat).
2. A blokk bekerül a P2P hálózatba és elterjed a csomópontok között.
3. A csomópontok leellenőrzik a blokk érvényességét (érvényes aláírások, nincs duplaköltés, stb.).
4. Ha a blokk érvényes és a konszenzus-mechanizmus szabályai teljesülnek, a blokkot hozzáadják a lánchoz.

Minden blokk a következő három alapvető elemet tartalmazza:

- Az előző blokk hash-e – ez köti össze a blokkokat egy láncba, és garantálja, hogy a korábbi blokkok tartalmát nem lehet megváltoztatni anélkül, hogy az összes következő blokk hash-e is megváltozna.
- N darab tranzakció – a blokk hasznos tartalma.
- Valamilyen bizonyíték (proof) – amely a konszenzus-mechanizmustól függ, és amelynek előállítása garantálja, hogy a blokkot a szabályok szerint hozták létre.

A blokkok láncolása biztosítja a rendszer integritását: ha valaki megpróbálna módosítani egy korábbi tranzakciót, az megváltoztatná annak a blokknak a hash-ét, ami érvényteleníténé az összes utána következő blokkot. Ezért egy támadónak az összes rákövetkező blokkot újra kellene számolnia – ami, mint látni fogjuk, a konszenzus-mechanizmus miatt gazdaságilag vagy számítástechnikailag megvalósíthatatlan.

1.2 Proof-of-work (munkabizonyíték)

A proof-of-work (PoW) az első és legismertebb konszenzus-mechanizmus, amelyet a Bitcoin alkalmaz. A lényege a következő: a bányász csomópontok együtt próbálják megtalálni az SHA-256 hash függvény egy speciális bemenetét, amely egy adott feltételt kielégít. Formálisan: keresünk egy nonce értéket, amelyre

$$H(\text{blokk-fejléc} \| \text{nonce}) < \text{target}$$

ahol H az SHA-256 hash függvény, a target pedig egy előre meghatározott küszöbérték. Mivel a hash függvény kimenetét nem lehet előre megjósolni (a hash függvény egyirányú és pszeudovéletlen kimenetet ad), az egyetlen módszer a nonce szisztematikus próbálgatása (brute force). A megoldás tehát a nonce érték, amellyel ha kiszámoljuk a blokk hash-ét, az kielégíti a nehézségi feltételt.

A nehézség (difficulty) beállítása kulcsfontosságú a rendszer stabilitásához. Minél nagyobb a nehézség (azaz minél kisebb a target), annál több próbálkozásra van szükség a megfelelő nonce megtalálásához, tehát annál lassabban születik új blokk. A Bitcoin esetében a nehézséget minden 2016 blokkonként (körülbelül két hetente) automatikusan úgy állítják be, hogy a blokkidő átlagosan 10 perc maradjon. Ha a bányászok száma (és ezzel a hálózat összesített számítási kapacitása, az ún. hashrate) megnő, a nehézséget emelik; ha csökken, csökkentik. Ez biztosítja, hogy a blokkok keletkezésének üteme nagyjából konstans maradjon, függetlenül attól, hány bányász dolgozik éppen a hálózatban.

Amikor egy bányász megtalálja a megfelelő nonce-t – vagyis a hash kielégíti a feltételt –, az új blokk készen áll: elküldi a hálózatnak, a többi csomópont ellenőrzi (ami triviálisan gyors, hiszen egyetlen hash-számítás), és ha érvényes, hozzáfűzik a lánchoz. A bányász jutalmul kap néhány frissen kibocsátott kriptovalutát (block reward) és az adott blokkba foglalt tranzakciók díjait (transaction fees). Ez az a pénzügyi ösztönző, amely motiválja a bányászokat az energiaigényes számítási munka elvégzésére.

A proof-of-work biztonsága azon alapul, hogy a számítási munka költséges, de az ellenőrzés olcsó. Egy támadónak, aki felül akarja írni a lánc történetét, az összes becsületes bányász együttes számítási kapacitásánál többel kellene rendelkeznie – ez az 51%-os támadás feltétele. Amíg a becsületes csomópontok rendelkeznek a hashrate többségével, a leghosszabb (legtöbb munkát tartalmazó) lánc mindig az ő kezükben lesz, és a támadók alternatív valósága nem tud felülkerekedni.

1.3 Proof-of-stake

A proof-of-stake (PoS) a proof-of-work alternatívája, amely a számítási munka helyett gazdasági letétre alapozza a konszenzust. A működési elv a következő: a résztvevők (validátorok) kriptovalutát fagyasztanak be („stake-elnek”) a láncon, ezzel jelezve, hogy érdekükben áll a rendszer helyes működése. A validátorokat a befagyasztott összeg

arányában választják ki blokkok javaslására és szavazásra – minél nagyobb valakinek a stake-je, annál nagyobb valószínűséggel kerül sorra.

A biztonság logikája analóg a PoW-val, de a „munka" helyett a „tőke" a szűk erőforrás: egy támadónak a teljes befagyasztott mennyiség 51%-át kellene megszereznie ahhoz, hogy a hálózatot manipulálhassa. Mivel a stake-elt valuta értéke a lánc biztonságától függ, egy sikeres támadás éppen azt az eszközt értéktelenítené, amelyet a támadó felhalmozott – ez erős gazdasági elrettentő hatást jelent. Emellett a legtöbb PoS rendszerben az a validátor, aki szabályellenesen viselkedik (pl. két különböző blokkra szavaz egyszerre), büntetésként elveszíti a befagyasztott tőkéjének egy részét vagy egészét (ezt hívják „slashing"-nek).

A blokkgyártásért és a szavazásért jutalom jár: a validátorok tranzakciós díjakat és/vagy újonnan kibocsátott kriptovalutát kapnak, ami ösztönzi a részvételt. A PoS előnye a PoW-val szemben, hogy nem igényel hatalmas energiafogyasztást, hiszen nincs szükség tömeges hash-számításra. Hátránya, hogy a „gazdagok gazdagabbak lesznek" problémát veti fel (minél több stake-ed van, annál több jutalmat kapsz), és a „nothing-at-stake" probléma is felmerül: egy validátornak elvileg nem kerül semmibe egyszerre több elágazásra szavazni, szemben a PoW-val, ahol a bányász számítási kapacitása véges. Ezt a modern PoS rendszerek (pl. Ethereum) a slashing mechanizmussal kezelik.

1.4 Egyéb konszenzus mechanizmusok

A proof-of-work és a proof-of-stake mellett számos más konszenzus-mechanizmus is létezik, amelyek különböző kompromisszumokat kínálnak a decentralizáció, a hatékonyság és a biztonság között.

A delegált proof-of-stake (DPoS) esetében a token-tulajdonosok nem közvetlenül vesznek részt a blokkgyártásban, hanem szavazataikkal validátorokat választanak, akik a nevükben járnak el. Ez növeli a hatékonyságot (kevesebb validátor, gyorsabb konszenzus), de csökkenti a decentralizációt. A DPoS-t alkalmazza például az EOS [1] (21 választott blokk-producer) és a Tron [2] (27 szuper-képviselő). Más megközelítések közé tartozik a Proof of Burn (kriptovaluta megsemmisítése a részvételi jog megszerzéséért, ilyen például a Counterparty [3]), a Proof of Activity (PoW és PoS hibridje, a Decred [4] alkalmazza) és a Proof of Luck (véletlenszám-generátorokra épülő kiválasztás). A Stellar Consensus Protocol [5] a föderatív bizánci megegyezés (Federated Byzantine Agreement) elvén működik, ahol a csomópontok maguk választják meg, kiben bíznak.

1.4.1 Practical Byzantine Fault Tolerance (PBFT)

A PBFT algoritmus [6] közvetlenül a bizánci tábornokok problémáját oldja meg egy gyakorlati környezetben. Miguel Castro és Barbara Liskov 1999-es tanulmánya azért volt áttörés, mert korábban a bizánci hibatűrő protokollokat túl lassúnak tartották a valós alkalmazásokhoz. Castro és Liskov megmutatták, hogy egy hatékony, háromfázisú protokollal a probléma gyakorlatilag is megoldható.

Az algoritmus három fázisból áll: (1) pre-prepare – a kijelölt vezető (primary) sorrendszámot rendel a kliens kéréséhez és elküldi a többi replikának; (2) prepare – a replikák megerősítik, hogy látták a kérést az adott sorrendszámmal; (3) commit – ha egy replika elegendő prepare üzenetet kapott ($2f$ darabot, ahol f a maximálisan tolerált hibás csomópontok száma), véglegesíti a műveletet. A protokoll összesen $3f + 1$ replikát igényel f bizánci hiba tolerálásához.

Két alapvető tulajdonságot garantál: safety (a jóhiszemű csomópontok soha nem fogadnak el ellentmondó állapotot – ez akkor is teljesül, ha a hálózat tetszőlegesen lassú) és

liveness (a rendszer végül mindig előrehalad, feltéve hogy a hálózat részlegesen szinkron). Ha a primary hibásan viselkedik, a protokoll view change mechanizmussal új vezetőt választ.

Castro és Liskov a BFT-NFS rendszerrel demonstrálták a protokoll gyakorlati használhatóságát: egy bizánci hibatűrő NFS (hálózati fájlrendszer) implementáció teljesítménye mindössze 3%-kal volt lassabb, mint a nem replikált változat. Ez megcáfolta a korábbi feltételezést, miszerint a bizánci hibatűrés túl drága lenne a gyakorlatban. A PBFT állapotgép-replikálással (state machine replication) éri el a konszenzust: minden csomópont ugyanazt az állapotgépet futtatja, és a protokoll biztosítja, hogy mindenki azonos sorrendben hajtsa végre a tranzakciókat. A PBFT lett az alapja számos későbbi blokklánc-protokollnak, köztük a Hyperledger Fabric korai verzióinak és a Tendermint-nek.

1.4.2 Tendermint és az FLP lehetetlenség

Hogyan lehet bányászat nélkül konszenzusra jutni? A Tendermint protokoll [7] a bizánci tábornokok problémájának egy modern, gyakorlati megoldása. Validátorokat definiál, akik kriptovalutát allokálnak (stake-elnek) a szavazás előtt, és mindenki annyit szavazhat, amennyi kriptovalutát befagyasztott.

A Tendermint tervezésekor fontos elméleti korlátot kellett figyelembe venni: az FLP lehetetlenségi tételt [8] (Fischer, Lynch, Paterson, 1985), amely kimondja, hogy teljesen aszinkron rendszerben – ahol az üzenetek késleltetésére semmilyen felső korlát nincs – lehetetlen determinisztikus konszenzust elérni, ha akár egyetlen hibás csomópont is van.

Az FLP tétel modellje a következő: N folyamat aszinkron üzenetküldéssel kommunikál, minden folyamatnak van egy bemeneti értéke (0 vagy 1), és a protokollnak biztosítania kell, hogy a nem hibás folyamatok megegyezzenek egyetlen közös döntési értékben. A hibás folyamat egyszerűen megáll (crash failure) – nem küld hamis üzeneteket, csak leáll. Az üzenetrendszer garantálja, hogy minden elküldött üzenet végül megérkezik, de tetszőlegesen késleltetheti és átrendezheti az üzeneteket. A fő tétel (Theorem 1) kimondja: egyetlen ilyen konszenzus-protokoll sem teljesen helyes – vagyis minden protokollnak létezik legalább egy megengedett végrehajtása, amelyben soha nem születik döntés. A bizonyítás kulcsgondolata a bivalens konfiguráció fogalma: olyan állapot, amelyből még mindkét döntési érték (0 és 1) elérhető. A szerzők megmutatják, hogy (1) létezik bivalens kezdeti konfiguráció, és (2) bármely bivalens konfigurációból konstruálható olyan végrehajtás, amely soha nem lép ki a bivalens állapotokból – tehát sosem dönt.

Ez az eredmény nem azt jelenti, hogy a konszenzus a gyakorlatban megoldhatatlan, hanem azt, hogy a megoldásoknak valamilyen extra feltételezéssel kell élniük. Ebből következik, hogy minden működő konszenzus-protokollnak feltételeznie kell valamiféle szinkronitást (vagy véletlenszerűséget). A Bitcoin például felteszi, hogy a csomópontok végül szinkronba tudnak jutni (az üzenetek véges időn belül megérkeznek), és a 10 perces blokkidő kellően nagy ahhoz, hogy a blokkok elterjedjenek a hálózatban. A Tendermint részlegesen szinkron modellben működik: feltételez egy (ismeretlen, de véges) időkorlátot, amely után az üzenetek biztosan megérkeznek.

A Tendermint 1/3 arányú csalót képes tolerálni, és két alapvető garanciát nyújt:

- Safety: a lojális validátorok mindig konzisztens blokkokat követnek – soha nem véglegesítenek két ellentmondó blokkot ugyanarra a magasságra.
- Liveness: a rendszer sosem áll le véglegesen, mindig képes új blokkot termelni, feltéve hogy a hálózat végül szinkronba kerül.

A részvételért jutalmat ad a rendszer (tranzakciós díjak és/vagy infláció formájában), amit a validátorok kamatként foghatnak fel a befagyasztott tőkéjük után. Fontos kérdés, hogy mi történik, ha valaki nem szavaz: hogyan lehet megkülönböztetni a szándékos támadót az egyszerűen offline validátortól? A legtöbb rendszer erre a megoldása az, hogy a hosszú ideig inaktív validátorokat fokozatosan büntetik (csökkentik a stake-jüket), de enyhébben, mint az aktívan rosszindulatú viselkedést.

1.5 Proof-of-spacetime

A proof-of-spacetime (PoSt) és az ehhez szorosan kapcsolódó proof-of-space (PoSpace) olyan konszenzus-mechanizmusok, amelyek a számítási munka (PoW) helyett a tárterületet használják szűk erőforrásként. Az alapötlet: ha egy résztvevő bizonyítani tudja, hogy egy adott mennyiségű tárhelyet lefoglalt és fenntartja, akkor ezzel „szavazati jogot" kap a konszenzusban. Ez a megközelítés energiahatékonyabb a PoW-nál, mivel a merevlemezek folyamatos üzemeltetése nagyságrendekkel kevesebb áramot fogyaszt, mint a tömeges hash-számítás.

1.5.1 A proof-of-space alapötlete

A legegyszerűbb konstrukció a következő: megkérünk egy résztvevőt, hogy számoljon ki és tároljon le egy nagy táblázatot (például egy hash függvény inverziós tábláját). Bizonyos időközönként rákérdezünk bizonyos értékekre – ha a résztvevő gyorsan válaszol, azzal bizonyítja, hogy a táblázatot valóban letárolta. Ha rendezve tárolta el az értékeket, gyorsan tudja invertálni a hash függvényt, és ezzel igazolja, hogy elegendő tárhelyet allokált.

Ez a naiv megközelítés azonban hibás a Hellman-féle time-memory trade-off [9] miatt. Hellman 1980-ban megmutatta, hogy ha N méretű a problématér (pl. egy N elemű táblázat teljes letárolása), akkor nem szükséges az egész táblázatot eltárolni: $T \cdot S = N$ összefüggés áll fenn, ahol T a számítási idő és S a tárolt adatmennyiség. A két szélsőség: teljes táblázat letárolása ($S = N$, $T = 1$) vagy teljes újraszámolás ($S = 1$, $T = N$). A kettő között Hellman megmutatta, hogy $N^{2/3}$ nagyságrendű tárhellyel és $N^{2/3}$ nagyságrendű számítással is megoldható a probléma. Ez azt jelenti, hogy egy naiv proof-of-space rendszerben a résztvevő sokkal kevesebb tárhelyet használhatna a szükségesnél, és a hiányzó adatokat menet közben számolná ki – tehát a bizonyíték nem lenne megbízható.

1.5.2 Hellman trade-off és a DES példa

Hellman az eredeti cikkében a DES titkosítóval illusztrálta a módszert. A DES egy szimmetrikus blokktitkosító, amely a következő leképezést valósítja meg:

$$\mathrm{DES} : \{0,1\}^{56} \times \{0,1\}^{64} \to \{0,1\}^{64}, \quad c = \mathrm{DES}(k, p)$$

ahol k az 56 bites kulcs, p a 64 bites nyílt szöveg, c pedig a 64 bites titkosított szöveg. A támadó ismeri a (p, c) párt, és a célja megtalálni a k kulcsot. A kulcstér mérete $N = 2^{56}$.

Hellman módszere a következő lépésekből áll. Definiálunk egy redukciós operátort $R : \{0,1\}^{64} \to \{0,1\}^{56}$, amely a titkosított szövegből kulcsot állít elő (például elhagyja az első 8 bitet). Ezzel konstruálunk egy $f : \{0,1\}^{56} \to \{0,1\}^{56}$ függvényt:

$$f(k) = R(\mathrm{DES}(k, p))$$

Ez a függvény a kulcstérből a kulcstérbe képez. Az előszámítási fázisban:

1. Generálunk m darab véletlenszerű kiindulási kulcsot: $k_1^{(0)}, k_2^{(0)}, \ldots, k_m^{(0)}$.
2. Minden kiindulási értékre t-szer alkalmazzuk az f függvényt, így m darab láncot kapunk:
$$k_i^{(0)} \xrightarrow{f} k_i^{(1)} \xrightarrow{f} k_i^{(2)} \xrightarrow{f} \cdots \xrightarrow{f} k_i^{(t)}$$
3. Csak a láncok elejét és végét tároljuk el: $(k_i^{(0)}, k_i^{(t)})$, rendezve $k_i^{(t)}$ szerint.

A tárigény tehát $S = 2m$ (kezdő- és végpont párok).

A keresési fázisban adott az ismeretlen k^* kulcshoz tartozó titkosított szöveg $c^* = \mathrm{DES}(k^*, p)$. A keresés:

1. Számoljuk $y_0 = R(c^*)$, majd $y_1 = f(y_0)$, $y_2 = f(y_1)$, ...
2. Minden y_j-re ellenőrizzük, hogy egyezik-e valamelyik tárolt végponttal $k_i^{(t)}$.
3. Ha $y_j = k_i^{(t)}$ valamely i-re, akkor a lánc elejéről $(k_i^{(0)})$ újraszámoljuk: $f^{(t-j-1)}(k_i^{(0)})$ adja a keresett kulcsot k^*.

Legfeljebb t darab y_j értéket számolunk ki $(y_0, y_1, \ldots, y_{t-1})$, mert a tárolt láncok hossza t. Az ötlet a következő: ha k^* valamelyik tárolt lánc $(t-j-1)$-edik pozíciójában van (azaz $k^* = k_i^{(t-j-1)}$), akkor az f függvényt j-szer alkalmazva $R(c^*)$-ra pontosan a lánc végpontjába $(k_i^{(t)})$ jutunk. Például ha k^* a lánc utolsó előtti eleme, akkor $y_0 = R(c^*)$ rögtön egyezik egy végponttal (1 lépés); ha k^* kettővel a vége előtt van, akkor $y_1 = f(y_0)$-nál találjuk meg (2 lépés); és így tovább, egészen addig, amíg ha k^* a lánc legelső eleme, akkor y_{t-1}-nél lesz ütközés (maximum t lépés). Ha t lépés után sincs egyezés, k^* nincs benne egyetlen tárolt láncban sem. A keresés legfeljebb t lépést igényel (ennyi f-alkalmazás). Az összefüggés: a láncok $m \cdot t$ különböző kulcsot fednek le (ideális esetben), tehát ha $m \cdot t = N$, a teljes kulcstér le van fedve. Az idő-memória kompromisszum:

$$T \cdot S = N, \quad \text{ahol } T = t \text{ (keresési idő)}, \quad S = m \text{ (tárolt láncok száma)}$$

Az optimális egyensúly $m = t = N^{1/2}$ esetén $T = S = N^{1/2}$, de a láncok közötti ütközések (collisions) miatt a gyakorlatban $m = t = N^{2/3}$ választás adja a legjobb kompromisszumot, $T \cdot S^2 = N^2$ korlát mellett.

1.5.3 Beyond Hellman: modern proof-of-space konstrukciók

A modern proof-of-space protokollok [10] kifejezetten úgy vannak tervezve, hogy ellenálljanak a Hellman-féle trade-off-nak. A cikk fő eredménye a következő: ha két véletlen függvényt kompozícióban alkalmazunk, az invertálás bizonyíthatóan nehezebb lesz, mint egyetlen függvény esetén. A konstrukció két véletlen függvényt használ: $f : [N] \to [N]$ és $g : [N] \times [N] \to [N]$. A kompozíciót a következőképpen definiáljuk:

$$g_f(x) = g(x, x') \quad \text{ahol } f(x) = f(x') \text{ és } x \neq x'$$

Vagyis g_f kiértékeléséhez meg kell találni egy $x' \neq x$ "ütköző" párt, amelyre $f(x) = f(x')$ – és ez az f invertálását igényli. A becsületes bizonyító letárolja f és g_f teljes függvénytábláját (összesen $\approx 2N \log N$ bit), és így bármely y értékre gyorsan megadja az (x, x') párt, amelyre $g_f(x) = y$.

A [10] cikk fő tétele a következő alsó korlátokat bizonyítja az invertáláshoz szükséges S tárhelyre és T lekérdezésre:

Hellman eredeti korlátai:

- Permutációra: $S \cdot T \in \tilde{O}(N)$, azaz $S = T \approx N^{1/2}$ elegendő (egyenlet (1)).
- Véletlen függvényre (Fiat–Naor): $S^2 \cdot T \in \tilde{O}(N^2)$, azaz $S = T \approx N^{2/3}$ elegendő (egyenlet (2)).
- Tetszőleges függvényre: $S^3 \cdot T \in \tilde{O}(N^3)$, azaz $S = T \approx N^{3/4}$ elegendő (egyenlet (3)).

A Beyond Hellman tétel alsó korlátja a g_f kompozícióra:

$$S^2 \cdot T \in \Omega(N^2) \quad \text{(alsó korlát, nem támadás!)}$$

Ez azt jelenti, hogy bármely támadónak, aki S bit segédinformációval és legfeljebb T lekérdezéssel akarja invertálni g_f-et, teljesítenie kell $S^2 \cdot T \geq \Omega(N^2)$-t. Például ha $S = T$, akkor $S = T = \Omega(N^{2/3})$ szükséges – ez megegyezik a véletlen függvények elleni legjobb ismert támadással, tehát elvileg ha komponáljuk attól nem csökken a biztonság szintje.

A konstrukció továbbá beágyazással (nesting) erősíthető: $k-1$ szintű beágyazás esetén az alsó korlát $S^k \cdot T \in \Omega(N^k)$-ra javul, ami $S = T = \Omega(N^{k/(k+1)})$-et ad. Ahogy k nő, ez közelít az $S = \Omega(N)$ vagy $T = \Omega(N)$ ideális korláthoz (vagyis vagy mindent letárolsz, vagy mindent újraszámolsz).

A gyakorlatban ez egy nagyjából 100 GB-os fájl („plot”) generálását jelenti, ami körülbelül 5 perc, ha 400 GB RAM áll rendelkezésre. A kulcskérdés az, hogy mennyi memória és CPU kellhet ahhoz, hogy a letárolás nélkül tudjuk invertálni a táblát. A fenti alsó korlátok garantálják, hogy a menet közbeni kiszámítás lényegesen drágább, mint a tárolás.

Hivatkozások

[1] Block.one, *EOS.IO Technical White Paper v2*, 2018. `https://github.com/EOSIO/Documentation`

[2] J. Sun, *TRON: Advanced Decentralized Blockchain Platform*, 2018. `https://tron.network/static/doc/white_paper_v_2_0.pdf`

[3] Counterparty Foundation, *Counterparty: Protocol Specification*, 2014. `https://counterparty.io`

[4] C. Yocom-Piatt, *Decred: An Autonomous Digital Currency*, 2016. `https://docs.decred.org/`

[5] D. Mazières, *The Stellar Consensus Protocol: A Federated Model for Internet-level Consensus*, Stellar Development Foundation, 2015.

[6] M. Castro, B. Liskov, *Practical Byzantine Fault Tolerance*, Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI), pp. 173–186, 1999.

[7] J. Kwon, *Tendermint: Consensus without Mining*, 2014. `https://tendermint.com/static/docs/tendermint.pdf`

[8] M. J. Fischer, N. A. Lynch, M. S. Paterson, *Impossibility of Distributed Consensus with One Faulty Process*, Journal of the ACM, 32(2):374–382, 1985.

[9] M. E. Hellman, *A Cryptanalytic Time-Memory Trade-Off*, IEEE Transactions on Information Theory, 26(4):401–406, 1980.

[10] H. Abusalah, J. Alwen, B. Cohen, D. Khilko, K. Pietrzak, L. Reyzin, *Beyond Hellman's Time-Memory Trade-Offs with Applications to Proofs of Space*, Advances in Cryptology – ASIACRYPT 2017, Lecture Notes in Computer Science, vol. 10625, Springer, pp. 357–379, 2017.